arXiv:2405.15216v1 [cs.LG] 24 May 2024

Denoising LM: Pushing the Limits of Error Correction Models for Speech Recognition

Zijin Gu, Tatiana Likhomanenko, He Bai, Erik McDermott, Ronan Collobert, Navdeep Jaitly
Apple
{zgu26,antares,hbai22,erik_mcdermott,collobert,njaitly}@apple.com

Abstract

Language models (LMs) have long been used to improve results of automatic speech recognition (ASR) systems, but they are unaware of the errors that ASR systems make. Error correction models are designed to fix ASR errors, however, they showed little improvement over traditional LMs mainly due to the lack of supervised training data. In this paper, we present Denoising LM (DLM), which is a *scaled* error correction model trained with vast amounts of synthetic data, significantly exceeding prior attempts meanwhile achieving new state-of-the-art ASR performance. We use text-to-speech (TTS) systems to synthesize audio, which is fed into an ASR system to produce noisy hypotheses, which are then paired with the original texts to train the DLM. DLM has several *key ingredients*: (i) up-scaled model and data; (ii) usage of multi-speaker TTS systems; (iii) combination of multiple noise augmentation strategies; and (iv) new decoding techniques. With a Transformer-CTC ASR, DLM achieves 1.5% word error rate (WER) on *test-clean* and 3.3% WER on *test-other* on Librispeech, which to our knowledge are the best reported numbers in the setting where no external audio data are used and even match self-supervised methods which use external audio data. Furthermore, a single DLM is applicable to different ASRs, and greatly surpassing the performance of conventional LM based beam-search rescoring. These results indicate that properly investigated error correction models have the potential to replace conventional LMs, holding the key to a new level of accuracy in ASR systems.

1 Introduction

Language models (LMs) have long been used to rerank hypotheses produced by automatic speech recognition (ASR) systems. In order to do so, the acoustic score of each hypothesis from the ASR is blended with the LM score for the hypothesis. However, LMs are trained on text corpora and are agnostic to the mistakes made by ASR. Thus significant work has gone into developing techniques to train ASR systems that incorporate an existing LM, so that the combined system works well.

Error correction models, on the other hand, are trained specifically to correct errors from ASR models, mapping hypotheses ("noisy" text from the ASR system) to the clean text, thus it sidesteps the problem of learning the correct weighting between the LM and the ASR. This can be an especially useful strategy when the ASR model has limited capacity and cannot produce structured output – for example with non-autoregressive ASR models. In those cases, a sequence-to-sequence error correction model can turn approximately correct outputs into structured outputs. Further, it can also take the text normalization role, simplifying the overall pipeline for ASR system.

One hurdle that arises is that error correction models typically require a large number of supervised training examples, but the number of utterances in a typical ASR dataset is comparatively small. In this paper, we propose **Denoising LM (DLM)**, an ASR error correction model that uses text-to-speech (TTS) systems to avoid this data scarcity issue. Text from a language model corpus can be fed into a

Preprint. Under review.

TTS system to produce audio, which is in turn fed into an ASR system to generate hypotheses. These hypotheses are then paired with the original text, to form the training dataset. The amount of training data can thus be scaled up arbitrarily by using a larger language corpus. Our key contributions are outlined below:

- To our knowledge, we are the first to make ASR error correction model state-of-the-art (SOTA) and show it can be significantly better than traditional neural LM. The proposed DLM achieves word error rate (WER) on LibriSpeech [34] – 1.4% (3.1%) on *dev-clean (other)* and 1.5% (3.3%) on *test-clean (other)*, without using any external audio data.
- We present DLM with its working recipe – there are several key ingredients that distinguish DLM from previous attempts and make error correction models work in practice:
  - We use multiple zero-shot, multi-speaker TTS systems to generate audio in different styles.
  - We mix real and synthetic data during training to keep the DLM grounded.
  - We combine multiple noise augmentation strategies such as frequency masking of spectral features and random substitution of characters of ASR hypotheses.
- The proposed DLM demonstrates good LM characteristics:
  - *Universality*: the same DLM is applicable to ASR systems of different architectures and denoises different datasets.
  - *Scalability*: the performance of DLMs improves with the increasing number of speakers, the size of the model, and the size of training text corpus.
  - *Efficiency*: DLMs can match traditional neural LM results by simply performing greedy decoding, with no need of heavy beam search decoding and rescoring.

The success of DLMs suggests that instead of complicated LM integration techniques, error correction models show more promising possibilities to bring the next level ASR accuracy.

2 Related Work

Finding better techniques to leverage LMs most effectively with neural acoustic models has been a long standing research problem [47, 44]. Earlier techniques back-propagated errors from LMs into neural ASRs using sequence discriminative criterion (e.g. Maximum Mutual Information) [23, 18, 47]. Later approaches attempted to merge features of text-only LMs with features of ASRs in either the shallower or deeper layers of the model [10, 41, 21].

Recently, error correction models, which post-process outputs from an ASR system to fix errors, have also been proposed [11, 39, 43, 50]. These approaches typically convert first-pass ASR hypotheses into cleaned up text. Even with just one best hypothesis from the ASR model, Transformer-based error correction models have shown improvements over the baseline acoustic models and acoustic models with n-gram language models [15, 52]. Further improvements can be made by using n-best ASR hypotheses as inputs to the clean up model as this provides more information [25, 27]; by using more advanced WER based metrics, instead of cross-entropy for model optimization [14]; or by using different variants that incorporate more compact inputs, such as phonemes [48, 7] and word lattices [28, 6]. More recently, large language models (LLMs) such as ChatGPT and LLaMA, were also shown to improve the transcription accuracy with their powerful linguistic representations [29, 37, 40].

However, the efficacy of these error correction approaches suffers due to the scarcity of the paired ASR output and ground truth transcriptions and *thus were not able to outperform conventional neural LMs integration yet.* As a result most clean up approaches start with a pretrained language model, e.g., BART [26], that is finetuned with the limited noisy ASR data [52, 7, 27]. Others incorporate different data augmentation strategies, such as SpecAugment [27] and dropout [15]. While these are helpful, the improvement may be limited by the data size. [11] have attempted to resolve this problem by using synthetic data for training error correction models on top of listen, attend and spell (LAS) [5] models. They show that error correction can improve results when combined with neural LMs, but error correction by itself is inferior to neural LM integration.

Another line of work is followed by [31, 30, 46]. These papers attempt to subtract the language model learned in the end-to-end ASR models, while integrating external LMs. Nevertheless, we believe that

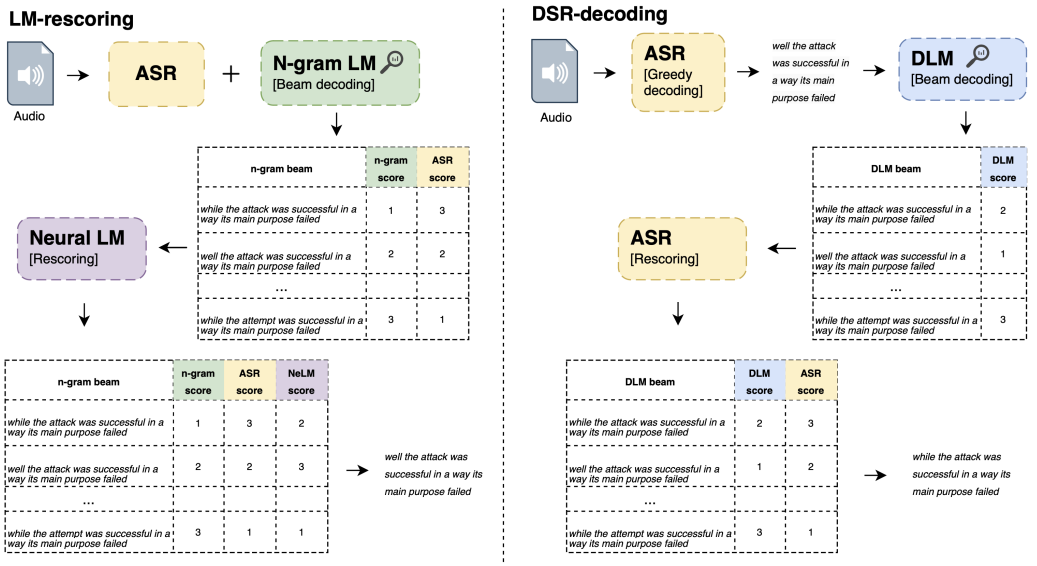

Figure 1: Conventional LM decoding and the proposed DSR-decoding. Left panel: given an input audio, an n-best list of the hypotheses from the beam-search decoding of the ASR model is generated with a N-gram word-level LM, which is then rescored using the neural LM scores and ASR scores. Right panel: given an input audio, ASR generates its greedy hypothesis, which is fed into DLM and it generates an n-best list of the corrected hypotheses, which are then rescored using both DLM scores and ASR scores.

any combination of independently designed ASR and LM models is inherently limited by the the ability to optimize the model subtraction, and the inherent lower expressive power of the combination method.

3 Denoising Speech Recognition (DSR)

To motivate error correction approach and our variant of it, we consider a probabilistic model $p_{\texttt{DSR}}(y|x)$, which we call a error correcting speech recognition (DSR) model. DSR is a cascade of two stochastic, discrete models – an ASR model which produces sequences $\tilde{y}$ of discrete tokens from audio input x with probability $p_{\texttt{ASR}}(\tilde{y}|x)$, and a DLM which transforms an input sequence, $\tilde{y}$, into an output sequence y with probability $p_{\texttt{DLM}}(y|\tilde{y})$. Under this model:

$$\log p_{\texttt{DSR}}(y|x) = \log \left(\sum_{\tilde{y}} p_{\texttt{DLM}}(y|\tilde{y}) p_{\texttt{ASR}}(\tilde{y}|x) \right). \tag{1}$$

Most error correction approaches optimize

$$\log p_{\texttt{DLM}}(y|\hat{y}), \quad \text{where } \hat{y} = \arg\max_{\tilde{y}} p_{\texttt{ASR}}(\tilde{y}|x), \tag{2}$$

which can be viewed as using an approximation to the lower bound of Equation (1)

$$\log p_{\texttt{DSR}}(y|x) \geq \sum_{\tilde{y}} p_{\texttt{ASR}}(\tilde{y}|x) \log p_{\texttt{DLM}}(y|\tilde{y}) \tag{3}$$

under the assumption that the ASR model is really peaked on a single transcript (the inequality comes from applying Jensen's inequality). Note, however, that using samples from the posterior of the model, $p_{\texttt{DSR}}(\tilde{y}|x, y)$, would be the correct distribution to optimize the above model on, but we (and prior works on error correction) do not follow that approach here. Variational approaches are alternative directions to consider under this framework which we leave as a future work.

3.1 Data generation

We would like to optimize Equation (2) in order to improve $\log p_{\texttt{DSR}}$. Given the unavailability of large amount of supervised data and the availability of a large corpus of text with distribution $p(y)$, we take the approach that we can learn a denoising distribution using a cascade of a TTS system and an ASR system to generate a reasonable distribution $\{(\hat{y}, y)\}$ to learn the error correcting distribution. The TTS system is used to create the distribution $\mathcal{D}_{\texttt{DSR}} = \{(\tilde{x}, y)\}$, where the audio $\tilde{x}$ is generated from the sentence y using the TTS system, and the ASR system is used to generate $\hat{y}$ from $\tilde{x}$. The dataset $\{(\hat{y}, y)\}$ is used to train a sequence-to-sequence model $p_{\texttt{DLM}}(y|\hat{y})$.

Unlike conventional LMs which place their probability mass on text sequences, these DLMs are conditional models whose input distribution reflects a corrupted version of the text distribution. Thus, it is critical to make sure that the corrupted distribution learned by the DLM is close to what is needed at test time. In this sense, TTS corrupted distribution is better than just randomly mutated text by itself [32],[1] because it captures the biases of the ASR system.

Different TTS systems give different corrupted text distributions. We found that good TTS systems need to be able to generate stylistic variations in audio (see Section 5.3). Besides TTS system, we show different useful strategies for introducing variant noise types to the text in Section 5.4. Finally, scaling the training text corpus size together with the number of parameters in the DLM is also beneficial (Section 5.3).

3.2 Decoding techniques

After both ASR and DLM models are trained, in order to decode an audio signal x to the DSR model, we seek the transcript y^*, which satisfies the following maximization:

$$y^* = \arg\max_{y} \sum_{\tilde{y}} p_{\texttt{DLM}}(y|\tilde{y}) p_{\texttt{ASR}}(\tilde{y}|x). \tag{4}$$

However, the exact optimization problem is computationally intractable. We consider one simplest greedy decoding, and also propose our novel decoding approach, which we call DSR-decoding.

Greedy-decoding In the simplest case, greedy-decoding follows two steps: 1) ASR takes in the audio x and outputs the greedy hypothesis $\hat{y}$; and 2) DLM takes in the noisy hypothesis $\hat{y}$ and do greedy decoding of it to y^*, agnostic to acoustic model probabilities:[2]

$$y^* = \arg\max_{y} p_{\texttt{DLM}}(y|\hat{y}); \;\; \hat{y} = \arg\max_{\tilde{y}} p_{\texttt{ASR}}(\tilde{y}|x), \tag{5}$$

where both maximizations are approximated by the greedy algorithm. Note that with this simple method, error correction could be performed without access to the audio or the ASR prediction scores, which is not achievable when using conventional LMs.

DSR-decoding To further enhance inference by incorporating acoustic model outputs alongside DLM, we propose a new variant of the decoding algorithm, dubbed *DSR-decoding*. It follows four steps: 1) ASR takes in the audio x and outputs the greedy hypothesis $\hat{y}$, same as in greedy-decoding; 2) DLM takes in the greedy hypothesis $\hat{y}$ and generates an n-best list of corrected candidates using a beam search; 3) For each candidate generated by DLM, we compute the acoustic score with ASR; 4) Each candidate is rescored using a weighted sum of the ASR score and the DLM score, using a single hyperparameter λ that is tuned on the entire validation set. In $\log$ space, the above can be written as:

$$y^* = \arg\max_{y \in n\text{-best}[p_{\texttt{DLM}}(.|\hat{y})]} \lambda \log p_{\texttt{DLM}}(y|\hat{y}) + \log p_{\texttt{ASR}}(y|x) \qquad \text{with} \qquad \hat{y} = \arg\max_{\tilde{y}} p_{\texttt{ASR}}(\tilde{y}|x), \tag{6}$$

where λ is a scalar that can be used to adjust the relative strengths of the ASR and the DLM and $\hat{y}$ is obtained by the greedy algorithm. Note that DSR-decoding is quite different from rescoring that is performed with neural LMs, and we show a comparison of DSR-decoding with conventional decoding approach in Figure 1. See Appendix D.1 for more details.

[1] As initial experiments we also tried carefully designed by hand corruptions, e.g. n-gram modifications of the text which did not work in practice.

[2] We experimented on CTC ASR models, so technically we do not use an exact maximization for $\hat{y}$. In practice we take an argmax of labels over label probabilities for each frame, deduplicate the inputs, and remove blank tokens. And we find that with this approximation DLM is able to generate good corrections.

4 Experimental Details

4.1 TTS systems and DLM training set

All our DLMs are trained on data derived from the LibriSpeech [34] LM corpus (40M sentences with a total of 800M words) if not stated otherwise. We use three different TTS systems to synthesize audio from this corpus in our experiments:

- **Tacotron-2** [38]. A single speaker high quality TTS system with a model from [1].
- **YourTTS** [4]. A zero-shot, multi-speaker TTS system with model from [1].
- **RichTTS**. An in-house autoregressive, zero-shot, multi-speaker TTS, similar in large part to Tacotron-2, but with additional inputs for speaker embeddings, using D-vectors [45]. This TTS is trained on the LibriSpeech ASR corpus.

For each sentence y in the text corpus we generate a corresponding audio waveform $\tilde{x}$ from the TTS system. For multi-speaker TTS systems, we randomly select a speaker from speakers in the LibriSpeech ASR training corpus. For YourTTS, this involves providing an audio of a randomly selected speaker as the conditioning example. For our internal RichTTS system, this is done by computing a D-vector from a random audio from a random speaker. ASR system is trained with Connectionist Temporal Classification [8] (CTC) loss. Then $\tilde{x}$ is processed with this ASR system to generate a hypothesis $\hat{y}$ that is paired with the sentence y. The entire dataset of $\{(\hat{y}, y)\}$ pairs is used to train the DLMs.

4.2 ASR models

In most experiments, ASR is a Transformer-based encoder model with 255M parameters. This ASR model is used throughout the paper to provide baseline WERs and we refer to it in the text as a *baseline ASR*.[3] We also experiment with a number of different ASR models to assess whether DLMs trained on noise generated by the *baseline ASR* would transfer to other ASR architectures. This includes Quartznet [24], Conformer [9] and Whisper [36]. We implemented our own Quartznet (7M params) and Conformer (102M params). Whisper models (Whisper-base, Whisper-small) are pretrained models [2] with a much larger proprietary audio corpus.

We train all ASR models (Transformer, Quartznet, and Conformer) on the LibriSpeech 960h train set with CTC loss and a character vocabulary, including apostrophe (‘). As is shown in prior work that TTS data can improve ASR models [17], we train several Transformer-based ASR models with different mixture proportions of LibriSpeech and TTS audio in a minibatch, and other settings remain the same as the *baseline ASR*. We denote these ASRs as *baseline ASR (LS+TTS)*. All training details are in Appendix D.2.

4.3 Denoising LMs and neural LMs

Denoising LM DLM is a Transformer-based model with a sequence-to-sequence encoder-decoder architecture trained with a cross entropy loss and a character vocabulary. There are 16 encoder layers and 4 decoder layers. Dropout and layer drop are set to 0.1. We vary the embedding dimension and the MLP hidden dimension for DLMs of different sizes: the embedding dimensions are 512, 768 and 1280, and MLP hidden dimensions are 2048, 3072 and 6144 for the 69M, 155M and 484M size DLMs respectively. The 484M architecture design was chosen based on reported results from [42]. Sinpos position embedding is used. The DLM training set is generated from LibriSpeech LM text corpus by different TTS systems but from a single early version *baseline ASR* model, which is trained with a smaller total batch size. Other details are in Appendix D.3.

Neural LM Neural LM is a Transformer-based model with a decoder-only architecture trained with a cross entropy loss and 10K word-piece vocabulary. It has 20 layers in the decoder, and shares the same embedding dimension and the MLP hidden dimension with DLM. Sinpos position embedding is used. Neural LMs are trained on LibriSpeech LM text corpus (the same text data are used for synthetic audio generation by TTS for DLM training). To performing neural LM rescoring (Figure 1 right panel), we generate an n-best list from the beam-search decoding of the CTC ASR model

[3] We used an earlier version of the *baseline ASR* with 2.6 (6.4)% WER on *test-clean (other)* to generate data.

Table 1: LibriSpeech validation and test sets word error rate (WER, %) for: i) Transformer ASR model (256M, *baseline*) with greedy decoding trained on LibriSpeech (960h), denoted as "baseline ASR", or on LibriSpeech (960h) and extra TTS data generated from LibriSpeech LM corpus by RichTTS, denoted as "baseline ASR (LS+TTS)" (mixing proportion is 1:1); ii) DLM (484M) with DSR-greedy/DSR-decoding and a neural LM (neLM, 500M) rescoring applied on top of ASR models. DLM is trained on data generated by the *baseline ASR* applied to LibriSpeech (960h) and YourTTS and RichTTS synthetic data (LibriSpeech LM corpus), used in proportion real:synthetic of 1:9 in the minibatch, with frequency masking and 10% random substitutions of characters.

Model	dev-clean	dev-other	test-clean	test-other
baseline ASR	2.1	5.5	2.2	5.3
+ neLM (LM-rescoring)	1.5	4.0	2.0	4.1
+ DLM (Greedy-decoding)	1.9	3.9	2.0	4.1
+ DLM (DSR-decoding)	1.5	3.4	**1.6**	**3.6**
baseline ASR (LS+TTS)	1.9	4.7	2.0	4.7
+ neLM (LM-rescoring)	1.3	3.4	1.8	3.7
+ DLM (Greedy-decoding)	1.8	3.6	1.9	3.9
+ DLM (DSR-decoding)	1.4	3.1	**1.5**	**3.3**

Table 2: Examples of ASR hypotheses, neural LM hypotheses and DLM corrections.

ASR	Neural LM	DLM	Error Type
but he'd bi'n too sick and too long a bed	but he'd been too sick and too long a bed	but he'd been too sick and too long abed	word boundary error
humanity was torred in her country	humanity was stored in our country	humanity was stirred in our country	spelling mistake
media too	media too	medea too	wrong word
the fact is the castle was much later than	the fact is the castle was much later than	the fact is the castle is much later than	grammatical error

with a 4-gram word-level LM trained on the same LibriSpeech LM text corpus with top-200k words, and rescore using the neural LM scores and CTC ASR scores, similar to [42]. Perplexity of trained models and other training details can be found in Appendix D.4.

5 Results

5.1 Pushing the limits: DLM achieves new SOTA WER on LibriSpeech

Our main results are in Table 1. We consider two ASR systems: one is the *baseline ASR* (Transformer CTC-based model with 255M parameters) trained on LibriSpeech; the other is the *baseline ASR (LS+TTS)* which is the same as *baseline ASR* but trained on LibriSpeech together with TTS audio data generated by RichTTS from the LibriSpeech LM corpus. The mixing ratio of LibriSpeech audio and TTS audio in a minibatch is 1:1, which leads to improved performance compared to *baseline ASR*. We show the other mixtures of LibriSpeech audio and TTS audio in a minibatch for *baseline ASR (LS+TTS)* in Appendix D.2 Table 8.

In Table 1, DLMs are trained on the mix of LibriSpeech audio and audio generated by YourTTS and RichTTS (total corpus is 2x) from LibriSpeech LM corpus. In both cases, DLM with greedy-decoding is comparable to neural LM with more expensive beam search and rescoring. DLM with DSR-decoding significantly outperforms a similar size neural LM with rescoring. Note that DSR-decoding does not necessarily increase the computational cost comparing with standard rescoring. In Table 2, we show error analyses of hypotheses obtained from ASR, neural LM and DLM. In these cases, neural LM fails to choose the correct answer from the beam of ASR but DLM is able to correctly identify the error and generate the correct transcription.

Our best combination comes from applying DLM to *baseline ASR (LS+TTS)*, which results in new SOTA WERs on LibriSpeech: 1.5% on *test-clean* and 3.3% on *test-other*. As shown in Table 3, our vanilla Transformer CTC-based ASR with DLM outperforms prior works with more sophisticated models and training criteria that do not use external audio data, and it even matches self-supervised models of the same size trained with additional 60k hours of unlabeled audio data.

Table 3: LibriSpeech test sets WER (%) for DLM and prior works that either do not use external audio data, or use self-supervised pretraining on Libri-Light [19] audio (LL-60k).

Prior work	External Data	test-clean	test-other
Transformer [42]	-	2.3	5.2
Context-Net(L) [12]	-	1.9	4.1
Conformer (Transducer) [9]	-	1.9	3.9
ASAPP-ASR [33]	-	1.8	4.5
E-branchformer + ILME [22]	-	1.8	3.7
SYNT++ [17]	-	2.4	6.3
LAS + SC + LM [11]	-	4.3	-
wav2vec 2.0-Large [3]	LL-60k	1.8	3.3
HUBERT-Large [16]	LL-60k	1.9	3.3
HUBERT-XL [16]	LL-60k	1.8	2.9
Conformer XXL [51]	LL-60k	1.5	3.1
baseline ASR (LS+TTS) + DSR-decoding (Ours)	-	**1.5**	**3.3**

5.2 DLMs are universal

A single DLM has capability of correcting errors for different ASRs and different datasets while outperforming the neural LMs. In Table 4, we show results of DLM trained to correct hypotheses from the *baseline ASR* on out-of-domain ASRs of different sizes and architectures: DLM not only improves over the ASR models but also consistently outperforms the same size neural LM. Also, the same DLM can be applied to Whisper models to further correct hypothesis generated by their default beam-search decoding. Note that Whisper models were trained on an entirely different and much larger proprietary data with word-piece tokens instead of characters; further Whisper is an encoder-decoder Transformer model with an architecture that is quite different from our CTC based *baseline ASR*.

Furthermore, we show that the DLM trained on the LibriSpeech LM corpus also improves results of the *baseline ASR* system when evaluating on different data. We use the TED-LIUM dataset [13], which is based on TeD talks, conversational in nature, and covers more contemporary topics. We observe around 16% WER improvement on dev and test set of TED-LIUM after apply DLM with DSR-decoding, which still outperforms neural LM (see Table 9 in Appendix E for details). Note that TED-LIUM is different from the read speech of the LibriSpeech, which corresponds to Gutenberg books, and hence the gains here are expected to be small.

5.3 DLMs are scalable

We train different DLMs and apply them to the *baseline ASR* to test the scalability of DLMs in three aspects: 1) scaling the number of speakers, 2) scaling the model size and 3) scaling the training dataset size.

Scaling the DLM size Table 5 (left) compares DLMs and neural LMs of different size: i) while model size increases the WER decreases; ii) DLMs outperforms neural LMs of the same size.

Scaling text corpus size To vary the text corpus used to generate the synthetic audio for DLM training, i) we randomly selected 20M sentences from the LibriSpeech LM corpus and formed 0.5x corpus (400M words); ii) we added in 40M sentences randomly selected from the Gutenberg dataset [49] and formed the 2x corpus (1.6B words). Table 5 (right) shows that DLMs benefits from more trainig data. We expect to see further gains from scaling to larger corpora in future work.

Scaling numbers of speakers To see the impact of the number of speakers used in audio generation, we obtained 20M unique sentences from 1, 10, 100, 1000, and 2000 speakers to form five training sets. We train five different DLMs (155M) on these data: using more speakers leads to lower WER, probably due to larger variability in types of errors generated, see Table 6. 100 speakers seems to be enough for Librispeech dev sets, but we keep using all speakers from LibriSpeech for the other experiments. Even if we use only one speaker, we can improve results by using a random example of their utterances for style-conditioning (see the last column in Table 6). This indicates that the

Table 4: LibriSpeech dev and test sets WER (%) of different ASR models trained on LibriSpeech (960h) with greedy decoding, with neural LM (neLM, 500M) rescoring or with DLM (484M) DSR-greedy/DSR-decoding. Both neural LM and DLM are the same as reported in Table 1.

Model	Size	dev		test	
		clean	other	clean	other
QuartzNet-CTC (Ours)	7M	6.4	17.0	6.5	16.9
+ neLM (LM-rescoring)		2.3	8.1	2.9	8.6
+ DLM (DSR-greedy)		2.7	8.1	2.8	8.3
+ DLM (DSR-decoding)		**2.1**	**6.9**	**2.3**	**7.1**
Conformer-CTC (Ours)	102M	2.4	5.7	2.6	5.6
+ neLM (LM-rescoring)		1.6	4.0	2.2	4.2
+ DLM (DSR-greedy)		1.9	4.0	2.2	4.1
+ DLM (DSR-decoding)		**1.5**	**3.4**	**1.7**	**3.6**
Whisper-base	74M	4.5	10.3	4.5	10.9
+ DLM (DSR-greedy)		2.9	7.1	3.1	7.9
Whisper-small	244M	3.6	7.1	3.3	7.6
+ DLM (DSR-greedy)		2.8	5.4	2.7	6.1

Table 5: LibriSpeech WER (%) for *baseline ASR* with (left) DLMs and neural LMs of different size with 800M words of text; (right) DLMs trained on different data size with 484M params. DSR-greedy (DSR-decoding) is reported for DLMs and rescoring for neural LMs. (left) YourTTS synthetic data, 1:9 real:synthetic data proportion, frequency masking and 10% random substitutions of characters are used; (right) RichTTS with 10% random substitutions of characters are used.

Size	dev-clean		dev-other	
	DLM	neLM	DLM	neLM
69M	2.1 (1.7)	1.8	4.8 (4.2)	4.6
155M	2.1 (1.6)	1.6	4.5 (4.0)	4.1
484M	2.0 (1.5)	1.6	4.3 (3.8)	4.0
1B	2.0 (1.5)	-	4.3 (3.7)	-

# of words	dev-clean	dev-other
0.5x (400M)	2.6 (1.8)	4.7 (3.8)
1x (800M)	2.2 (1.6)	4.3 (3.6)
2x (1.6B)	2.2 (1.6)	4.2 (3.5)

Table 6: LibriSpeech DSR-greedy (DSR-decoding) WER (%) for DLMs (155M) trained on RichTTS synthetic data (400M words) with 10% random substitutions and different # of speakers.

# of speakers	1	10	100	1000	2000	1 (w/ 100 speaker embeddings)
dev-clean	3.2 (2.0)	2.6 (1.8)	2.6 (1.8)	2.5 (1.8)	2.6 (1.8)	2.9 (1.9)
dev-other	5.3 (4.1)	4.9 (4.0)	4.7 (4.1)	4.7 (4.1)	4.7 (4.0)	5.1 (4.1)

variance from the same speaker's different embeddings can also provide a certain amount of noise, which is beneficial for the DLM to learn and correct.

5.4 Ablation: design proper error correction data distribution

In this section, we show the data construction path to our final results highlighting the importance to generate a variety of relevant error types (see Table 7, left). We first generate a dataset of 40M $\{(\hat{y}, y)\}$ pairs using YourTTS. Training on these data improves WER of ASR on *dev-other*, with slight degradation on *dev-clean* (row 2). We hypothesise that training data are still relatively clean and there are not many corrections for the model to learn. We experiment with adding different types of noise carefully such that the noise distribution does not shift the training distribution away from the test time distribution too significantly. First, we found that randomly substituting some percentage of the characters in the input transcript is beneficial (row 4) as long as it is not too high ($s = 10\%$ works quite well). Second, generating hypotheses from noisy audio, where the noise comes from masking frequency bands of the spectrogram input to the ASR model (2 masks with max width 30), in a manner similar to SpecAugment [35], also helps (row 5). To ground the data distribution of the DLM more closely to real data, we added a small portion (10%) of noisy hypotheses by running the ASR system on the LibriSpeech 960h data set. This gave us further gains on both *dev-clean* and *dev-other* sets (row 7).

We then repeated the audio generation process with RichTTS trained with LibriSpeech, and got a new set of data. We found that combining the data from two TTS systems led to better WER compared

Table 7: LibriSpeech WER (%) for DLMs (484M) trained on different data reported with DSR-greedy (DSR-decoding): s is a random character substitution rate; "FM" is a frequency masking; "real" refers to hypotheses from a real audio.

Model	dev-clean	dev-other
baseline ASR	2.1	5.5
+ DLM, Tacotron + $s = 10\%$	4.1 (2.0)	6.6 (4.8)
+ DLM (YourTTS)	2.3 (1.6)	4.9 (4.0)
+ $s = 10\%$	2.2 (1.5)	4.6 (3.8)
+ FM	2.3 (1.6)	4.6 (3.7)
+ RichTTS	2.1 (1.5)	4.2 (3.7)
+ FM + real	2.0 (1.5)	4.3 (3.8)
+ RichTTS + FM + real	1.9 (1.5)	3.9 (3.4)
+ DLM, RichTTS + $s = 10\%$	2.2 (1.6)	4.3 (3.6)

to using only one TTS system (row 6). Lastly, we mixed all the data construction tricks together, which produced the best results (row 8). Although not reported, we found that generating multiple synthetic audio samples using the same TTS system from the same sentence did not improve results, underscoring the importance of generating stylistic variations of noise for better error correction.

5.5 TTS audio is as good as real audio for DLMs

We explore whether there is a gap between the performance of DLMs trained on $\hat{y}$ obtained from synthetic audio vs from real audio. We use $\sim$60k hours of real labeled data from Libriheavy [20] which gives 400M transcribed words, and generate $\hat{y}$ from it using the *baseline ASR*. For a fair comparison, we also generate TTS data by RichTTS for the same sentences. For both datasets of $(\hat{y}, y)$ we train a DLM-real and DLM-synthetic. On the *dev-clean* set, the DLM-synthetic (WER 1.7%) is worse than the DLM-real (WER 1.5%). Surprisingly, on the *dev-other* set, DLM-synthetic (WER 3.7%) outperforms the DLM-real (WER 4.1%). Overall, the gap between DLM-synthetic and DLM-real is small and thus effect of TTS generated data is small for DLMs. See Table 10 in Appendix E for more details.

5.6 High quality TTS are not what you need

We assess the impact of using different TTS systems for generating DLMs training data. We compare Tacotron, YourTTS and RichTTS by evaluating ASR systems on the audio generated by these TTS models: synthetic audio from Tacotron (6%) has significantly lower WER than that from YourTTS (10%) and RichTTS (12%) when evaluated with Whisper-small. See Table 11 in Appendix E for more details. We then trained DLMs using data generated by the three TTS systems separately, and applied them to the *baseline ASR* model, see Table 7. Though Tacotron data has higher quality than YourTTS and RichTTS data, it performs poorly in terms of DLM error correction. We posit that there might not be enough meaningful noise in the Tacotron data for the error correction model to learn. In this sense, improving ASR systems in the proposed error correction paradigm is easy – it does not require high-quality TTS.

6 Conclusion

We present DLM, a scaled ASR error correction model that leverages synthetic data and learns the proper error distribution. For the first time, we show that state-of-the-art WER can be achieved with such error correction models on the well-known LibriSpeech benchmark, and that error correction models are able to outperform conventional neural language models by a large margin. Meanwhile, we provide a detailed investigations on how to design the right error correction data distribution, including mixing real data, using multiple TTS systems, and employing noise augmentation strategies. With comprehensive experiments, we demonstrate that the proposed DLMs are universal, scalable and efficient, showing great potential to replace neural LM in modern ASR systems. We further discuss limitations of our work in Appendix A and broader impact in Appendix B. The success of DLMs leads us to contend that the time is right to reimagine how large text corpora might improve ASR systems.

References

[1] Tacotron-2 and yourtts pretrained models. `https://github.com/coqui-ai/TTS`.

[2] Whisper pretrained models. `https://github.com/openai/whisper`.

[3] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, 33:12449–12460, 2020.

[4] Edresson Casanova, Julian Weber, Christopher D Shulby, Arnaldo Candido Junior, Eren Gölge, and Moacir A Ponti. Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone. In *International Conference on Machine Learning*, pages 2709–2720. PMLR, 2022.

[5] William Chan, Navdeep Jaitly, Quoc V Le, and Oriol Vinyals. Listen, attend and spell. *arXiv preprint arXiv:1508.01211*, 2015.

[6] Lingfeng Dai, Lu Chen, Zhikai Zhou, and Kai Yu. Latticebart: Lattice-to-lattice pre-training for speech recognition. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6112–6116. IEEE, 2022.

[7] Samrat Dutta, Shreyansh Jain, Ayush Maheshwari, Souvik Pal, Ganesh Ramakrishnan, and Preethi Jyothi. Error correction in asr using sequence-to-sequence models. *arXiv preprint arXiv:2202.01157*, 2022.

[8] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.

[9] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. In *Proc. Interspeech*, 2020.

[10] Caglar Gulcehre, Orhan Firat, Kelvin Xu, Kyunghyun Cho, Loic Barrault, Huei-Chi Lin, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. On using monolingual corpora in neural machine translation. *arXiv preprint arXiv:1503.03535*, 2015.

[11] Jinxi Guo, Tara N Sainath, and Ron J Weiss. A spelling correction model for end-to-end speech recognition. In *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5651–5655. IEEE, 2019.

[12] Wei Han, Zhengdong Zhang, Yu Zhang, Jiahui Yu, Chung-Cheng Chiu, James Qin, Anmol Gulati, Ruoming Pang, and Yonghui Wu. Contextnet: Improving convolutional neural networks for automatic speech recognition with global context. In *Proc. Interspeech*, 2020.

[13] François Hernandez, Vincent Nguyen, Sahar Ghannay, Natalia Tomashenko, and Yannick Esteve. Ted-lium 3: Twice as much data and corpus repartition for experiments on speaker adaptation. In *Speech and Computer: 20th International Conference, SPECOM 2018, Leipzig, Germany, September 18–22, 2018, Proceedings 20*, pages 198–208. Springer, 2018.

[14] Takaaki Hori, Chiori Hori, Shinji Watanabe, and John R. Hershey. Minimum word error training of long short-term memory recurrent neural network language models for speech recognition. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5990–5994, 2016.

[15] Oleksii Hrinchuk, Mariya Popova, and Boris Ginsburg. Correction of automatic speech recognition with transformer sequence-to-sequence model. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7074–7078. IEEE, 2020.

[16] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

[17] Ting-Yao Hu, Mohammadreza Armandpour, Ashish Shrivastava, Jen-Hao Rick Chang, Hema Koppula, and Oncel Tuzel. Synt++: Utilizing imperfect synthetic data to improve speech recognition. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7682–7686. IEEE, 2022.

[18] Navdeep Jaitly, Patrick Nguyen, Andrew Senior, and Vincent Vanhoucke. Application of pretrained deep neural networks to large vocabulary speech recognition. In *Proc. Interspeech*, pages 2578–2581, 2012.

[19] Jacob Kahn, Morgane Riviere, Weiyi Zheng, Evgeny Kharitonov, Qiantong Xu, Pierre-Emmanuel Mazaré, Julien Karadayi, Vitaliy Liptchinsky, Ronan Collobert, Christian Fuegen, et al. Libri-light: A benchmark for asr with limited or no supervision. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7669–7673. IEEE, 2020.

[20] Wei Kang, Xiaoyu Yang, Zengwei Yao, Fangjun Kuang, Yifan Yang, Liyong Guo, Long Lin, and Daniel Povey. Libriheavy: a 50,000 hours asr corpus with punctuation casing and context. *arXiv preprint arXiv:2309.08105*, 2023.

[21] Anjuli Kannan, Yonghui Wu, Patrick Nguyen, Tara N Sainath, Zhijeng Chen, and Rohit Prabhavalkar. An analysis of incorporating an external language model into a sequence-to-sequence model. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5828. IEEE, 2018.

[22] Kwangyoun Kim, Felix Wu, Yifan Peng, Jing Pan, Prashant Sridhar, Kyu J Han, and Shinji Watanabe. E-branchformer: Branchformer with enhanced merging for speech recognition. In *2022 IEEE Spoken Language Technology Workshop (SLT)*, pages 84–91. IEEE, 2023.

[23] Brian Kingsbury. Lattice-based optimization of sequence classification criteria for neural-network acoustic modeling. In *2009 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 3761–3764. IEEE, 2009.

[24] Samuel Kriman, Stanislav Beliaev, Boris Ginsburg, Jocelyn Huang, Oleksii Kuchaiev, Vitaly Lavrukhin, Ryan Leary, Jason Li, and Yang Zhang. Quartznet: Deep automatic speech recognition with 1d time-channel separable convolutions. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6124–6128. IEEE, 2020.

[25] Yichong Leng, Xu Tan, Rui Wang, Linchen Zhu, Jin Xu, Wenjie Liu, Linquan Liu, Xiang-Yang Li, Tao Qin, Edward Lin, et al. Fastcorrect 2: Fast error correction on multiple candidates for automatic speech recognition. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4328–4337, 2021.

[26] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020.

[27] Rao Ma, Mark JF Gales, Kate M Knill, and Mengjie Qian. N-best t5: Robust asr error correction using multiple input hypotheses and constrained decoding space. *arXiv preprint arXiv:2303.00456*, 2023.

[28] Rao Ma, Hao Li, Qi Liu, Lu Chen, and Kai Yu. Neural lattice search for speech recognition. In *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7794–7798. IEEE, 2020.

[29] Rao Ma, Mengjie Qian, Potsawee Manakul, Mark Gales, and Kate Knill. Can generative large language models perform asr error correction? *arXiv preprint arXiv:2307.04172*, 2023.

[30] Erik McDermott, Hasim Sak, and Ehsan Variani. A density ratio approach to language model fusion in end-to-end automatic speech recognition. *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 434–441, 2019.

[31] Zhong Meng, Sarangarajan Parthasarathy, Eric Sun, Yashesh Gaur, Naoyuki Kanda, Liang Lu, Xie Chen, Rui Zhao, Jinyu Li, and Yifan Gong. Internal language model estimation for domain-adaptive end-to-end speech recognition. In *2021 IEEE Spoken Language Technology Workshop (SLT)*, pages 243–250. IEEE, 2021.

[32] Mohammad Norouzi, Samy Bengio, Navdeep Jaitly, Mike Schuster, Yonghui Wu, Dale Schuurmans, et al. Reward augmented maximum likelihood for neural structured prediction. *Advances In Neural Information Processing Systems*, 29, 2016.

[33] Jing Pan, Joshua Shapiro, Jeremy Wohlwend, Kyu J Han, Tao Lei, and Tao Ma. Asapp-asr: Multistream cnn and self-attentive sru for sota speech recognition. In *Proc. Interspeech*, 2020.

[34] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: an asr corpus based on public domain audio books. In *2015 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 5206–5210. IEEE, 2015.

[35] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. In *Proc. Interspeech*, pages 2613–2617, 2019.

[36] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning*, pages 28492–28518. PMLR, 2023.

[37] Srijith Radhakrishnan, Chao-Han Huck Yang, Sumeer Ahmad Khan, Rohit Kumar, Narsis A. Kiani, David Gomez-Cabrero, and Jesper Tegnér. Whispering LLaMA: A cross-modal generative error correction framework for speech recognition. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[38] Jonathan Shen, Ruoming Pang, Ron J Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, Rj Skerrv-Ryan, et al. Natural tts synthesis by conditioning wavenet on mel spectrogram predictions. In *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4779–4783. IEEE, 2018.

[39] Prashanth Gurunath Shivakumar, Haoqi Li, Kevin Knight, and Panayiotis Georgiou. Learning from past mistakes: improving automatic speech recognition output via noisy-clean phrase context modeling. *APSIPA Transactions on Signal and Information Processing*, 8:e8, 2019.

[40] Gan Song, Zelin Wu, Golan Pundak, Angad Chandorkar, Kandarp Joshi, Xavier Velez, Diamantino Caseiro, Ben Haynor, Weiran Wang, Nikhil Siddhartha, et al. Contextual spelling correction with large language models. In *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 1–8. IEEE, 2023.

[41] Anuroop Sriram, Heewoo Jun, Sanjeev Satheesh, and Adam Coates. Cold fusion: Training seq2seq models together with language models, 2018.

[42] Gabriel Synnaeve, Qiantong Xu, Jacob Kahn, Tatiana Likhomanenko, Edouard Grave, Vineel Pratap, Anuroop Sriram, Vitaliy Liptchinsky, and Ronan Collobert. End-to-end ASR: from supervised to semi-supervised learning with modern architectures. In *ICML 2020 Workshop on Self-supervision in Audio and Speech*, 2020.

[43] Tomohiro Tanaka, Ryo Masumura, Hirokazu Masataki, and Yushi Aono. Neural error corrective language models for automatic speech recognition. In *Proc. Interspeech*, pages 401–405, 2018.

[44] Shubham Toshniwal, Anjuli Kannan, Chung-Cheng Chiu, Yonghui Wu, Tara N Sainath, and Karen Livescu. A comparison of techniques for language model integration in encoder-decoder speech recognition. In *2018 IEEE spoken language technology workshop (SLT)*, pages 369–375. IEEE, 2018.

[45] Ehsan Variani, Xin Lei, Erik McDermott, Ignacio Lopez Moreno, and Javier Gonzalez-Dominguez. Deep neural networks for small footprint text-dependent speaker verification. In *2014 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 4052–4056. IEEE, 2014.

[46] Ehsan Variani, David Rybach, Cyril Allauzen, and Michael Riley. Hybrid autoregressive transducer (hat). In *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6139–6143, 2020.

[47] Karel Veselý, Arnab Ghoshal, Lukáš Burget, and Daniel Povey. Sequence-discriminative training of deep neural networks. In *Proc. Interspeech*, pages 2345–2349, 2013.

[48] Haoyu Wang, Shuyan Dong, Yue Liu, James Logan, Ashish Agrawal, and Yang Liu. Asr error correction with augmented transformer for entity retrieval. In *Proc. Interspeech*, 2020.

[49] Qiantong Xu, Tatiana Likhomanenko, Jacob Kahn, Awni Hannun, Gabriel Synnaeve, and Ronan Collobert. Iterative Pseudo-Labeling for Speech Recognition. In *Proc. Interspeech*, pages 1006–1010, 2020.

[50] Shiliang Zhang, Ming Lei, and Zhijie Yan. Investigation of Transformer Based Spelling Correction Model for CTC-Based End-to-End Mandarin Speech Recognition. In *Proc. Interspeech*, pages 2180–2184, 2019.

[51] Yu Zhang, James Qin, Daniel S Park, Wei Han, Chung-Cheng Chiu, Ruoming Pang, Quoc V Le, and Yonghui Wu. Pushing the limits of semi-supervised learning for automatic speech recognition. *arXiv preprint arXiv:2010.10504*, 2020.

[52] Yun Zhao, Xuerui Yang, Jinchao Wang, Yongyu Gao, Chao Yan, and Yuanfu Zhou. Bart based semantic correction for mandarin automatic speech recognition system. *arXiv preprint arXiv:2104.05507*, 2021.

A Limitations

In our empirical results we reported only CTC-based ASR models. While our preliminary results show similar trends for encoder-decoder models, deeper assessment is further needed.

While we evaluated applicability of trained DLMs (for read speech and text from books) to out-of-domain speech data (evaluation on TED-LIUM with conversational speech) and other ASR systems (conv-based and Conformers), broader investigation on applicability to cases with domain shift is needed.

We assessed initial scaling up to 1.6B words in text and 1.3B model size, however further scaling experiments are needed with other datasets as performance on LibriSpeech dataset is already at human level.

As the main goal of the paper to show that error correction models can outperform neural LMs and potential for future developments of LMs in error correction direction, we did not work on efficiency of the inference as well as on the integration of DLMs with streaming models.

As all machine learning models, DLMs are trained on data which has bias: in our case the bias comes not only from the text data but also from the text to speech systems and real audio data. As we demonstrated, diversity of the generated data by text to speech systems (e.g. multi-speaker, different utterances for the same speaker, several text to speech systems) helps to reduce the final error of ASR systems, however further research is needed to alleviate other potential biases, discrimination, and privacy risks.

As we showed, DLMs can outperform neural LMs without the access to the real audio data while using large amount of synthetic data. On the one hand, this unlocks research for low resource languages where vast amount of audio is not accessible. On the other hand, it is possible that it reinforces biases of the text to speech systems into the ASR system, and further, it needs additional computational resources to generate synthetic data. We attempted to assess the bias introduced by the text to speech system by comparing it with the real data usage, where we see that careful design of text to speech system can reduce the introduced bias.

B Broader Impact

This paper presents a way to improve speech recognition by using text to speech systems. Improved ASR could lead to improved user experiences for people, especially those requiring assistive technologies. We do use zero-shot text to speech systems, which can be used for mis-information; however, zero-shot text to speech is not the theme of this paper, nor do we aim to release it as a part of this work. In our work, we use text to speech system is used solely to improve ASR system.

C Data, Pretrained Models, Code and Reproducibility

Whisper's code and models weights are released under the MIT license [2]. YourTTS and Tacotron-2 models and code are released under MPL-2.0 license [1].

LibriSpeech [34] dataset is released under CC BY 4.0, Libriheavy [20] dataset and its code are released under Apache-2.0 license, TED-LIUM v3 [13] is released under CC BY-NC-ND 3.0.

We do not plan to open source our in-house RichTTS model or its training code. We also do not plan to open source any generated data by any TTS model used in the paper.

We plan to open source the code of ASR and DLM training and inference as well as instructions how we generated data from YourTTS.

We tried to include as much information as possible about all models configurations, training recipes and extra experiments we did with their outcomes throughout the paper and in Appendix. We are working on open sourcing the code for ASR and DLM training and inference to speed up adoption of our research results.

Table 8: LibriSpeech dev sets WER (%) for Transformer *baseline ASR* models (255M) trained with a mixture of real (LibriSpeech train data) and synthetic (generated from LibriSpeech LM text corpus by RichTTS) audio data. "real:synthetic" denotes the proportion (%) of corresponding data in a minibatch.

real:synthetic	dev-clean	dev-other
0:100	12.7	33.4
30:70	2.1	5.3
50:50	1.9	4.7
70:30	2.0	4.9
100:0	2.1	5.5

D Training Details

All models are optimized on the validation data of LibriSpeech by adjusting the learning rate and optimizer. For the final results we evaluated all trained models on the test sets of LibriSpeech.

D.1 LM-rescoring v.s. DSR-decoding

In conventional neural LM rescoring (e.g. [42]), the ***ASR generates a beam*** (possibly with a weak n-gram LM), which is rescored using the neural LM.[4] For the exact details of rescoring with neural LM we refer the reader to [42]. In contrast, here, ***DLM generates a beam***, using only the greedy output of the ASR and the ASR scores are then used to rescore the DLM beam by blending them with the DLM scores. We adopted nucleus sampling with threshold 0.9 to ensure we get full length beams. In terms of computational cost (for beam generation, scores evaluation and re-ranking), the rescoring with neural LMs and DSR-decoding approaches are similar, while DLM greedy error correction is cheaper as it does not involve the beam search at all.

D.2 ASR models

The *baseline ASR* is a Transformer-based encoder model with 255M parameters: 1D convolution of kernel 7 and stride 3 followed by absolute sinusoidal positional embedding and 36 transformer blocks with pre-LayerNorm, an embedding dimension of 768, 4 heads, 3072 units in the MLP layers, dropout of 0.1 and layer drop of 0.1. The model is trained with CTC loss and a character vocabulary, including apostrophe ('). This ASR model is used throughout the paper to provide noisy transcripts for DLMs training.

We implemented our own Quartznet and Conformer, trained also with a CTC loss and a character vocabulary, including apostrophe ('). Conformer model has 102M parameters and consists of 1D convolution of kernel 7 and stride 3 followed by 16 conformer blocks with an embedding dimension of 512, 4 heads, 2048 units in the MLP layers, and dropout of 0.1.

All acoustic models are fed with 80 channel log-mel filterbanks, with a 25ms sliding window strided by 10ms. SpecAugment [35] is used as augmentation with 2 frequency masks with max width 30 and 10 time masks with max width 50 and ratio 0.1. We pack total 3.56h, 4.44h and 4.44h of audio via dynamic batching for Transformer, Conformer and Quartznet, respectively. The gradient clipping is set to 1.0 / 0.5 / 1.0, while learning rate warmup is performed for 64k / 10k / 64k steps, with a peak learning rate of 0.001 / 0.002 / 0.0022 for the Transformer / Conformer / Quartznet models. For Transformer and Conformer, a cosine learning rate decay schedule is applied followed by a step decay of rate 0.5, while for Quartznet we use exponential learning rate decay instead. All models are trained with the AdamW optimizer with 1e-6 weight decay until the greedy WER stops improving on the validation sets (*dev-clean, dev-other*).

In addition, we train *baseline ASR* with the mixture of LibriSpeech 960h train data (real) and synthetic audio data generated by RichTTS model from the LibriSpeech LM text corpus. The training setup is the same as for the *baseline ASR* trained on real data only, but we vary the mixture proportion of real and synthetic audio in the minibatch. Results are shown in Table 8.

[4] Alternatively, joint decoding is also possible, using ASR model and neural LM together in the decoding, but it is more expensive and in practice the oracle WER of the first pass n-gram decoding is already very low.

Table 9: LibriSpeech dev and test sets WER (%) of *baseline ASR* model trained on LibriSpeech (960h) with greedy decoding, with neural LM (neLM, 500M) rescoring or with DLM (484M) DSR-greedy/DSR-decoding. Both neural LM and DLM are the same as reported in Table 1. We also report WER (%) on dev and test sets of out-of-domain TED-LIUM.

Model	Size	dev		test		TED-LIUM	
		clean	other	clean	other	dev	test
Transformer ASR (LS)	256M	2.1	5.5	2.2	5.3	11.6	10.8
+ neLM (LM-rescoring)		1.5	4.0	2.0	4.1	10.6	9.7
+ DLM (DSR-greedy)		1.8	3.9	2.0	4.1	11.0	10.8
+ DLM (DSR-decoding)		**1.5**	**3.4**	**1.6**	**3.6**	**9.7**	**9.2**

D.3 DLMs

DLMs are trained with a dynamic batch size of total 160,000 tokens and a character vocabulary. We use AdamW optimizer with weight decay 0.01, gradient clipping of 0.1 and learning rate warmup of 64k steps, followed by a constant learning rate of 0.001 for 300k steps, then a step decay with rate of 0.5 and step size of 200k until WER of DLM-greedy reaches plateau on validation sets (*dev-clean, dev-other*).

In all experiments we use exactly the same training hyperparameters changing only model size and / or data on which DLMs are trained.

D.4 Neural LMs

For neural LM training, we use a 10K word-piece vocabulary created from the LibriSpeech train data. The models are trained on LibriSpeech LM text corpus (the same text data are used for synthetic audio generation by TTS for DLM training) where tokenized texts are concatenated together, separated by the end of sentence (EOS) token and batched into total 768 examples with a window of 256 tokens (total 196,608 tokens). Neural LMs are trained with AdamW optimizer with $1e-8$ weight decay and cosine learning rate schedules with a peak learning rate of 0.001 for 500k steps. Warmup is performed 16k steps. Perplexity for 69M, 155M and 484M LMs are 34.45, 31.49 and 30.9 respectively.

D.5 Compute

All models are trained on A100 with 80GB memory with 1 node of 8 GPUs for 3-5 days. Some models we train with larger batch size which results in longer training with gradient accumulation.

For data generation by YourTTS model we use CPU inference and it takes 100k CPU hours (as it is not batched version of public code). With our in house RichTTS (116M), we spent 12,800 GPU hours to synthesize 40M audios (roughly 137k hours of data).

DSR-decoding with beam of 64 is sufficient, while the beam of 10 gives similar results within 0.1% variation in WER), which takes about 10 mins to finish on 8 A100 GPU for dev sets.

E Results

In Table 9 we show results for *baseline ASR* when neural LM and DLM are applied on top for both LibriSpeech and TED-LIUM data. Note, that for TED-LIUM evaluation both ASR model and neural LM / DLM are out-of-domain as trained on LibriSpeech audio data and LibriSpeech LM text corpus, emulating both domain shifts: in acoustics and linguistics.

Comparison between usage real audio and synthetic audio for DLM data generation is given in Table 10 and between TTS systems is given in Table 11. For the later, we first generate audio data by TTS models from the text data of LibriSpeech *dev-clean* and *dev-other* and then evaluate quality of ASR models in WER on these audio data.

Table 10: LibriSpeech WER (%) for DLMs (484M) reported with DSR-greedy (DSR-decoding): DLM-synthetic is using audio generated by RichTTS from Libriheavy transcriptions, DLM-real is using real audio of Libriheavy for $\hat{y}$ generation by *baseline ASR*. Both DLMs are trained with 10% random characters substitutions.

Model	dev-clean	dev-other
baseline ASR	2.1	5.5
+ DLM-synthetic	2.4 (1.7)	4.5 (3.7)
+ DLM-real	2.0 (1.5)	4.6 (4.1)

Table 11: Whisper and *baseline ASR* WER (%) on audio generated from LibriSpeech dev sets by different TTS models.

TTS System	Whisper-small		Baseline ASR	
	dev-clean	dev-other	dev-clean	dev-other
Tacotron	5.9	5.2	6.8	6.2
YourTTS	8.2	9.8	8.6	10.5
RichTTS	7.7	11.9	9.5	16.7

Table 12: Key differences between DLMs and spelling correction models from [11].

	DLM	SC	Reference
TTS	multi-speaker	single-speaker	Table 7
Augmentation	text substitution frequency masking multiple TTS real data	reverberation background noise environmental noise	Table 7
Decoding	DSR-decoding	LM-rescoring	Table 1

Table 13: Oracle WER (%) in the beam for DLM (484M) and neLM (500M) reported in Table 1

Model	dev-clean	dev-other	test-clean	test-other
DLM	0.36	1.29	0.46	1.33
neLM	0.24	1.52	0.69	1.60

E.1 Other empirical findings

Encoder and decoder layers We investigate what part of the model is more important in the error correction model: encoder or decoder. With decreasing the number of encoder layers and increasing the number of decoder layers while the total number of layers is constant the WER degrades, thus showing that DLMs should have deep encoders and shallow decoders.

Role of tokenization As preliminary experiments we compared word-piece tokenization vs character one for DLMs training. However, word-piece one underperforms a lot character-based, thus all final results are reported with characters tokenization. This might be due to that our ASR is char-based so its errors are mostly on character level.

Applicability to ASR trained on 100h We also conducted similar experiments on ASR trained on LibriSpeech 100h which has 6% (18%) WER on *dev-clean (other)*. We observe over 50% WER reduction by simply applying a 180M DLM without the need of any design of the training data. Error

correction for weaker ASR is overall much easier to obtain improvements comparing with an already powerful ASR.

Oracle WER in the beam Oracle WER in the beam for our best DLM and neLM is reported in Table 13.

E.2 Pushing the limits of DLM: comparison with prior works

Using TTS data for ASR error correction is not new, as prior work [11] has made such attempts to Listen, Attend and Spell (LAS) models. However, it is worth noting that the improvement from their spelling correction (SC) model is marginal, even when applying to a relatively weak ASR system. Besides some fundamental differences like the ASR type and tokenization, we outline several key ingredients in Table 12 that distinguish us from this prior work and *are crucial to push the limits of DLMs to outperform neural LMs.*